

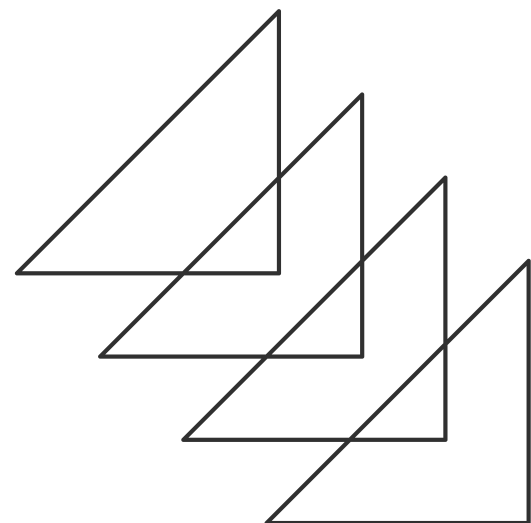
**GROUP 06**

*Module 02*

# **DART ESSENTIALS**



***PRM393 – Mobile Programming***



# TABLE OF CONTENTS

PAGE 03

DART STRUCTURE

PAGE 16

FUNCTIONS

PAGE 09

DATA TYPES

PAGE 18

OOP

PAGE 13

COLLECTIONS

PAGE 20

NULL SAFETY

PAGE 14

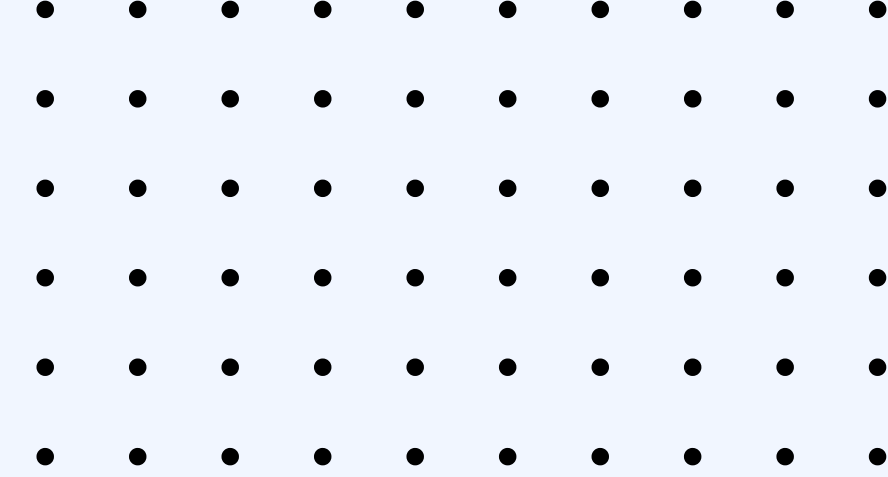
CONTROL FLOW

PAGE 23

ASYNC  
PROGRAMMING

01.

# WHAT IS DART?



A client-optimized, class-based language that created and maintained by Google for developing fast apps on any platform.

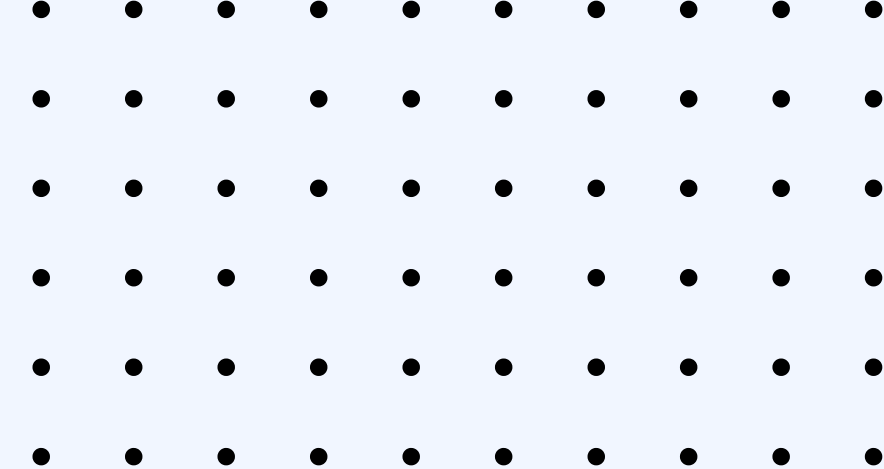


To offer the most productive programming language for  
**multi-platform development**,  
paired with a **flexible execution runtime platform** for app frameworks.

The **Flutter framework** is a popular, multi-platform UI toolkit that's powered by the Dart platform, and that provides tooling and UI libraries to build UI experiences that run on iOS, Android, macOS, Windows, Linux, and the web.

# 01.

# WHAT IS DART?



Dart uses a unique dual-compilation strategy to optimize the developer experience and the end-user experience:

- **JIT (Just-In-Time): Used during development.**

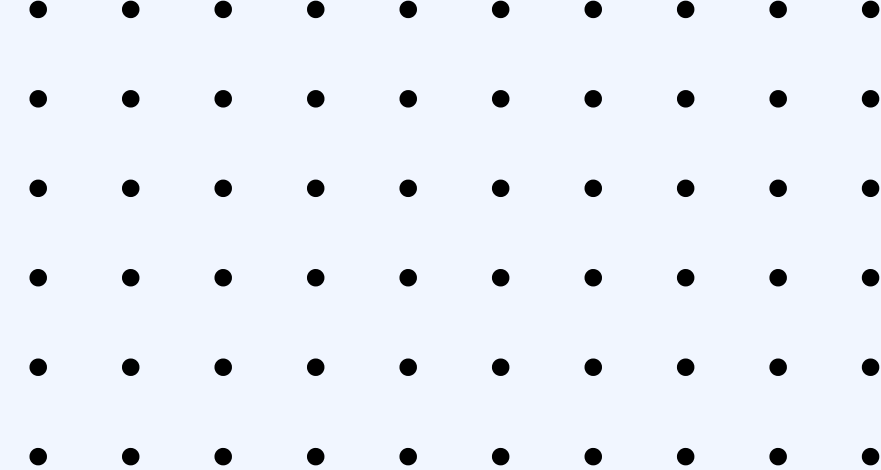
Enables Stateful Hot Reload (see changes in < 1 second without restarting the app).

- **AOT (Ahead-Of-Time): Used for production.**

Compiles to native machine code (ARM/x64) for lightning-fast startup and smooth 60fps/120fps performance.

# 01.

# WHAT IS DART?



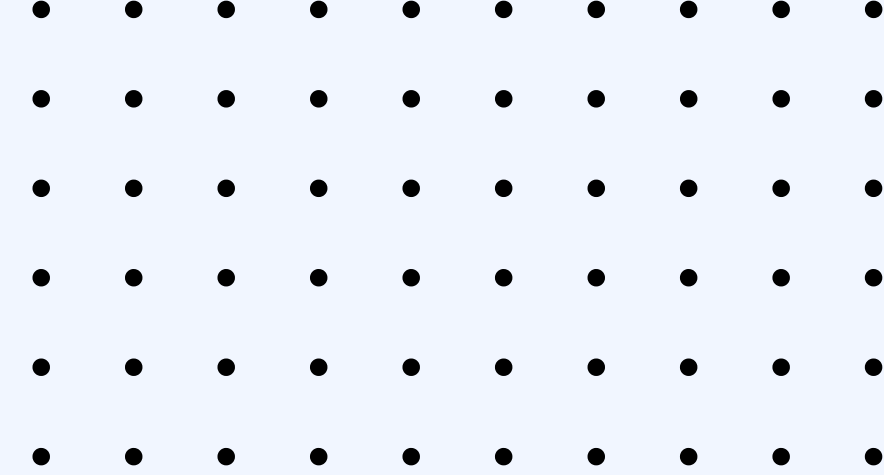
Dart is designed to handle UI interactions and data fetching simultaneously without "jank" (stuttering).

- Futures: Represents a value that will be available later.
- Async/Await: Readable syntax for non-blocking code.
- Streams: For handling sequences of data (e.g., real-time chat updates).

```
Future<String> fetchUserRole() async {  
  // Simulating a network request  
  var role = await database.getRole();  
  return "Role: $role";  
}
```

01.

# WHAT IS DART?



| Feature        | Dart                             | JavaScript                 | Kotlin                    |
|----------------|----------------------------------|----------------------------|---------------------------|
| Type System    | Strong (Static & Sound)          | Dynamic (Weak)             | Strong (Static)           |
| Concurrency    | Isolate-based (No shared memory) | Event Loop (Single-thread) | Coroutines (Multi-thread) |
| Main Use Case  | Cross-platform UI (Flutter)      | Web Development            | Android / JVM             |
| Learning Curve | Gentle (familiar to C#/Java)     | Low/Moderate               | Moderate                  |

Dart vs. JavaScript vs. Kotlin

# 02.

## PROGRAM STRUCTURE

Every Dart program starts at the `main()` function.  
`print()` writes a line to the console.

```
void main() {  
    print("Hello Dart");  
} // Output: Hello Dart
```

# 03.

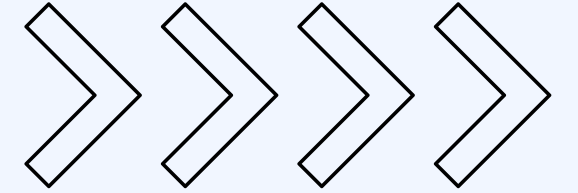
## DATA TYPES

Core types: `int`, `double`, `String`, `bool`, `List`, `Set`, `Map`  
Strong typing with generics (`List<int>`, `Map<String, int>`)

```
int age = 20;  
double gpa = 3.75;  
String campus = "FPT HCM";  
bool pass = true;
```

# DATA TYPES

| Type     | Description                                    |
|----------|------------------------------------------------|
| Numbers  | int (whole numbers), double (fractional)       |
| String   | Text enclosed in single or double quotes       |
| bool     | Boolean value: true or false                   |
| List     | Ordered collection of elements                 |
| Set      | Unordered collection of unique elements        |
| Map      | Key–value pairs                                |
| null     | Represents absence of a value                  |
| Object   | Base class for all Dart objects                |
| enum     | Fixed list of constant values                  |
| Iterable | Abstract collection type used in loops         |
| Never    | For functions that never return (always throw) |
| Dynamic  | Can hold any type; resolved at runtime         |
| Void     | Indicates no return value                      |



# 04.

## VARIABLES & MEMORY

- Inference: var is for inferred types, while dynamic is for runtime-resolved types.
- Constants:
  - final: Runtime-constant (assigned once).
  - const: Compile-time constant.

```
var name = "FPT University";  
final year = 2024;  
const pi = 3.14159;  
dynamic x = 10; x = "Hello";
```

# 05.

# OPERATORS

Operators allow you to perform mathematical, comparison, and logical operations.

Arithmetic: +, -, \*, /, %.

Comparison: >, <, >=, <=, ==.

Logical: && (AND), || (OR), ! (NOT).

```
int a = 10; int b = 3;  
print(a + b); // 13  
print(a / b); // 3.3333333333333335  
print(a ~/ b); // 3  
print(a % b); // 1
```

```
print(a > b); // true  
print(a == 10); // true
```

```
bool isTrue = (a > b) && (b == 3);  
print(isTrue); // true  
print(!isTrue); // false
```

06.

# STRING INTERPOLATION

- Concept: The ability to embed variables or expressions directly inside a string using the \$ symbol.
- Usage: Use **`$variableName`** for simple variables and **`${expression}`** for complex logic.

```
String campus = "FPT HCM";  
print("Welcome to $campus"); // Result: Welcome to FPT HCM  
print("Name length: ${campus.length}"); // Result: 7
```

# 07.

## COLLECTIONS

- **List:** An ordered collection (array) that allows duplicate elements.
- **Set:** An unordered collection of unique elements (duplicates are ignored).
- **Map:** A collection of key–value pairs, where each key is unique.

```
List<int> nums = [1, 2, 2]; // Result: [1, 2, 2]
```

```
Set<int> uniqueNums = {1, 2, 2, 3}; // Result: {1, 2, 3}
```

```
var user = {"id": 1, "name": "Nam"};  
print("User name: ${user['name']}"); // Result: Nam
```

08.

# CONTROL FLOW

**If/Else:** Executes blocks of code based on a boolean condition.

```
int score = 80;  
if (score >= 50) print("Pass"); else print("Fail");
```

**Switch:** Evaluates an expression against multiple discrete cases;  
more readable than nested if-else.

```
var day = "Mon";  
switch(day) {  
  case "Mon": print("Work"); break;  
  default: print("Rest");  
}
```

09.

# LOOPS

For loop: Classic counter-based loop.

```
for(int i=0; i<3; i++) {  
    print(i);  
}
```

For-in: The standard way to iterate over every item in a collection.

forEach: A functional approach to perform an action on every element

```
for (var x in [1, 2, 3]) print(x);  
[1, 2, 3].forEach((e) => print(e));
```

10.

# FUNCTIONS

Structure: Functions define reusable blocks of code with parameters and return types.

Arrow Syntax ( $\Rightarrow$ ): Used as a shorthand for functions that contain only one expression.

```
// Standard
```

```
int add(int a, int b) { return a + b; }
```

```
// Arrow Syntax
```

```
int addShort(int a, int b)  $\Rightarrow$  a + b;
```

10.

# FUNCTIONS

Named Parameters: Defined inside { }. They make function calls clearer and allow parameters to be optional.

Default Values: You can provide a fallback value if the parameter is not passed.

```
void greet({String name = "Guest"}) {  
    print("Hello $name");  
}
```

```
greet(); // Output: Hello Guest  
greet(name: "Minh"); // Output: Hello Minh
```

# 11.

# INTRO TO OOP

## Classes & Objects

**Class:** A blueprint for creating objects (defining fields and methods).

**Object:** An instance of a class.

**Constructors:** Used to initialize objects; Dart offers concise this syntax to assign values directly.

```
class Car {  
  String brand;  
  Car(this.brand); // Concise constructor  
  void start() => print("Started: $brand");  
}  
  
var myCar = Car("Toyota"); // Object creation
```

# 11.

# INTRO TO OOP

## Classes & Objects

**Inheritance (extends):** Allows a class to share behavior from another class.

**Override (@override):** Used to redefine a method from the parent class to change its behavior.

```
class Animal {  
    void sound() => print("Some sound");  
}
```

```
class Dog extends Animal {  
    @override  
    void sound() => print("Woof"); // Custom behavior  
}
```

12.

# NULL SAFETY

- To prevents "Null Pointer Exceptions" by catching them during coding, not at runtime.
- Operators:
  - ?: Declares a variable that can be null.
  - ??: Fallback operator (uses a default value if the variable is null).
  - !: Assertion operator (forces the compiler to treat a value as non-null).

```
String? name; // Nullable  
print(name ?? "Guest"); // Fallback to "Guest"
```

# 13. COLLECTION (MAP & FILTER)

**Map:** Transforms every element in a collection into something else.

**Where (Filter):** Keeps only the elements that meet a specific condition.

```
final numbers = [1, 2, 3, 4, 5];
```

```
final squares = numbers.map((n) => n * n).toList();  
// [1, 4, 9, 16, 25]
```

```
final evens = numbers.where((n) => n.isEven).toList();  
// [2, 4]
```

# 14.

# EXCEPTIONS

**Try/Catch:** Blocks used to catch runtime errors so the app doesn't crash.

**Finally:** A block that always runs, whether an error occurred or not (used for cleanup).

```
try {  
    int result = 10 ~/ 0; // Risky code  
} catch (e) {  
    print("Error: $e"); // Handle error  
} finally {  
    print("Done"); // Always runs  
}
```

# 15.

# ASYNCHRONOUS PROGRAMMING

**Future:** Represents a single value that will be available later (like an API response).

**Async/Await:** Syntax that allows you to write asynchronous code as if it were synchronous/sequential.

**Streams:** A sequence of asynchronous events emitted over time (e.g., a timer or chat messages)

```
Future<String> fetch() async {  
    await Future.delayed(Duration(seconds:1));  
    return "Done";  
}  
  
void main() async {  
    print(await fetch());  
}
```

16.

# STREAMS

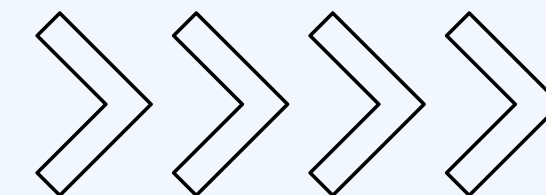
// Example 1: periodic stream (single-subscription)

```
final stream = Stream.periodic(const Duration(milliseconds: 400), (i) => i)
    .take(5); // 0..4 then complete
```

```
stream.listen(
    (value) => print('Value: $value'),
    onDone: () => print('Done'),
);
```

// Example 2: transform & filter

```
final evens = stream.where((n) => n.isEven).map((n) => n * 10);
evens.listen((v) => print('Even*10: $v'));
```



**THANK YOU**

*for your listening!*

